Qwen

Architecting Narrative-Free Combat: A Blueprint for Engine-Agnostic Systems Using ECS and Gameplay Tags

Foundational Architecture: Data-Oriented Design and Entity-Component-Systems

The primary objective of constructing a combat system that is simultaneously engine-agnostic and free of embedded lore necessitates a foundational architectural shift away from conventional Object-Oriented Programming (OOP) paradigms [17]. The provided research materials converge on a powerful alternative: Data-Oriented Design (DOD), often implemented through an Entity-Component-System (ECS) pattern [15] [30]. This approach fundamentally reorients software development from a model centered on objects and their behaviors to one focused on the efficient processing of data [27]. By separating identity, pure data, and behavior into distinct concepts, ECS provides the structural rigor required to create a system that is highly modular, performant, and entirely driven by external configuration rather than hard-coded logic [16] [19]. This section will explore the core tenets of DOD and ECS, detailing how their principles directly address the dual requirements of portability and narrative neutrality. It will analyze the performance benefits derived from optimized memory layouts, explain the composition-over-inheritance philosophy, and demonstrate how this architecture serves as the essential bedrock for a truly data-driven combat system.

The cornerstone of the ECS paradigm is the deliberate separation of three distinct elements: Entities, Components, and Systems [16]. An 'Entity' is the simplest of the trio, serving merely as a unique identifier or a lightweight handle, devoid of any data or behavior [15] [17]. Its sole purpose is to act as a container or a link, grouping together a collection of components. For example, a single `Entity` ID could represent a player character, an enemy grunt, and a destructible crate, with their unique characteristics defined solely by the components attached to them. This contrasts sharply with OOP, where each of these would typically be a distinct class inheriting from a base `GameObject` class, leading to complex and rigid inheritance hierarchies [17]. A key advantage of ECS is its ability to handle complex compositions without such hierarchies;

a flying enemy with an inventory, for instance, is created simply by adding a `CanFly` component and an `Inventory` component to an entity, avoiding the combinatorial explosion of classes seen in traditional OOP [15].

Components are the second pillar and represent the "what" an entity has [15]. They are pure data structures, often referred to as Plain Old Data (POD), containing only state information and no associated functions or behavior [17, 19]. Each component defines a single aspect of an object's state. For a combat system, this means having distinct components like `HealthComponent`, `PositionComponent`, `VelocityComponent`, `WeaponComponent`, and `AbilityComponent` [18]. A `HealthComponent` might contain fields for `current` and `max` health points, while a `PositionComponent` holds the entity's coordinates [18]. This strict separation of data from behavior is fundamental to achieving the goal of zero lore embedding. The code does not dictate *how* an entity takes damage; it merely defines a `HealthComponent`. The logic for modifying this component resides elsewhere, in a system. This allows designers to change an entity's capabilities at runtime simply by adding, removing, or altering components, a process that requires no modification to the source code itself [15]. For example, rendering an enemy invulnerable would involve simply removing its `HealthComponent`, a dynamic change that is difficult to achieve with static OOP inheritance.

The third and final element is the 'System', which represents the "what" an entity does [15]. Systems are the functional units of the ECS, containing all the game logic and behavior [19]. A system operates globally on all entities that possess a specific set of required components [16]. For instance, a `MovementSystem` would query the ECS for all entities that have both a `PositionComponent` and a `VelocityComponent`, and then iterate through this collection, updating each entity's position based on its velocity [19]. Similarly, a `DamageApplicationSystem` would target entities with a `HealthComponent` and listen for a damage event, applying the specified damage by modifying the relevant component's data [41]. This structure inherently promotes code reusability; the same `MovementSystem` can be applied to a projectile, a player character, and an NPC, as long as they all conform to the required data signature of having position and velocity [17]. This focus on processing collections of data over time is the essence of Data-Oriented Design, which prioritizes performance by optimizing how the CPU accesses and processes data in memory [30, 37].

One of the most compelling arguments for adopting an ECS/DOD architecture is its profound impact on performance, particularly concerning CPU cache efficiency [15, 30]. Modern CPUs execute instructions millions of times faster than they can fetch data from

main memory (RAM). Therefore, the speed of a program is often limited not by the complexity of its calculations, but by how efficiently it can move data between RAM and the CPU's much faster on-chip caches (L1, L2, etc.) [30]. DOD is an approach to structuring code around how data flows and is transformed to optimize this process [30]. The central concept is "data locality," which dictates that data that is used together should be stored together in contiguous blocks of memory [30] [37]. When an application needs to access a piece of data, the CPU loads a whole "cache line" (typically 64 bytes) from RAM into the cache. If the rest of the data in that line is also useful for subsequent operations, those operations can be performed at lightning speed from the cache. If not, valuable cache space is wasted on irrelevant data, leading to more frequent and costly trips back to RAM.

Traditional OOP often leads to poor data locality. In an OOP hierarchy, an object's data members may not be stored contiguously in memory, especially if they are allocated on the heap via pointers [37]. Iterating over a collection of heterogeneous game objects and accessing a single field (e.g., health) across all of them results in scattered, non-sequential memory accesses, causing numerous cache misses [29]. ECS and DOD combat this issue head-on by reorganizing data for sequential access. The most basic layout is Array-of-Structs (AoS), where an array stores structs for a single object type, grouping all of that object's fields together [28]. While intuitive, AoS can still be inefficient for certain operations. The optimal approach for performance-critical systems like combat is often Structure-of-Arrays (SoA), where data is organized by field across all entities [28] [29]. In an SoA layout, there would be one large array for all `PositionComponent.x` values, another for all `y` values, and so on [17]. A system operating on positions would then iterate through these arrays sequentially, resulting in perfect cache-line utilization [29]. This layout is also highly amenable to Single Instruction, Multiple Data (SIMD) vectorization, where a single CPU instruction can operate on multiple data points simultaneously, dramatically accelerating computations like physics updates or particle simulations [15] [28].

Modern ECS frameworks implement sophisticated variations of SoA to balance performance with flexibility. Archetype-based systems, used by engines like Unity DOTS and Bevy, group entities into dense tables where each table corresponds to a unique combination of components ("archetypes") [18]. All entities within a given archetype table have the exact same set of components, and each component type forms a column in the table [18]. This allows systems to iterate over a specific component type across all relevant entities with exceptional speed and cache efficiency, while also enabling constant-time access to an entity's full component set [18]. These systems often employ deferred execution models, where changes like adding or removing components are staged and

processed after a frame's update loop completes. This prevents iteration inconsistencies and maintains the integrity of the tightly packed memory layout during active processing [18]. Benchmarks comparing such designs against traditional OOP have shown significant performance gains, with one study reporting an average frame rate increase from 36.13 FPS (OOP) to 65.81 FPS (archetype SoA ECS) on a modern CPU [18]. This performance focus is not just an optimization; it is a core tenet of DOD, ensuring that the chosen architecture can scale to handle thousands of combatants without degrading simulation quality [15].

While ECS is a powerful pattern, it is crucial to understand that it is an architectural style, not a tool tied to a specific engine [30]. One can implement the core principles of ECS—separating data and behavior, using composition over inheritance, and organizing data for cache efficiency—without relying on a framework like Unity's DOTS or Unreal's Mass [27]. The fundamental building blocks are simple: custom arrays to store component data and functions (systems) that iterate over these arrays to perform logic [27]. For example, instead of a formal ECS framework, a developer could maintain several parallel arrays: one for `EntityId`, one for `HealthComponent` structs, one for `PositionComponent` structs, and so on. A movement system would then iterate through these arrays in lockstep. This approach reinforces the engine-agnostic nature of the design, as it relies only on standard language features like arrays and structs, making it trivial to port to a different platform or integrate with a different rendering or physics library. Even complex game mechanics can be modeled purely as data. A persistent ability, such as a continuous laser beam or a DoT (Damage over Time) effect, can be represented by storing its state (like remaining duration or cooldown) directly within its own component, allowing the same system logic to handle both transient projectiles and long-lasting effects uniformly [20]. This deep integration of data modeling into the very fabric of the game's logic is a hallmark of a mature DOD/ECS architecture and is precisely what enables a combat system to be both flexible and entirely divorced from narrative text or hardcoded story beats.

Decoupling Through Abstraction: Dependency Injection and Interface-Based Programming

Achieving complete engine-agnosticism and maintaining a clean separation of concerns within a combat system are critically dependent on robust decoupling strategies. No component of the system, from a low-level animation controller to a high-level combat

rule, should have direct knowledge of another's concrete implementation. This principle is most powerfully enforced through interface-based programming and the use of Dependency Injection (DI) [4] [5]. These practices work in concert to invert the flow of control in the application, allowing modules to depend on abstract contracts rather than concretions, thereby insulating the core gameplay logic from both other game systems and the volatile specifics of the underlying hardware or software platforms [22] [24]. This section will delve into the mechanics of DI and the guiding principle of Dependency Inversion, explaining how they form the backbone of a modular, testable, and portable architecture. It will detail the various injection methods, discuss the strategic placement of dependencies, and contrast this approach with more monolithic initialization patterns.

Dependency Injection is a programming technique where an object does not create its dependencies but receives them from an external source [5]. This separates the creation and usage of an object's dependencies, leading to loosely coupled code [5]. Instead of a `PlayerCombatComponent` instantiating a `DamageCalculator` internally with a `new DamageCalculator()` call, it declares that it *needs* a `IDamageCalculator` service. This interface, `IDamageCalculator`, defines the contract—the methods available—without exposing the details of the implementation [4]. An "injector," which could be a dedicated service locator, a factory, or a specialized DI container, is responsible for finding or creating a concrete `DamageCalculator` instance and providing it to the `PlayerCombatComponent` [5]. This simple inversion of responsibility yields profound benefits. Firstly, it enhances testability. During unit testing, the `PlayerCombatComponent` can be instantiated with a "mock" or "stub" implementation of `IDamageCalculator` that simulates responses without performing actual calculations, allowing tests to run quickly and in isolation from real dependencies like databases or file systems [4] [5]. Secondly, it facilitates parallel development, as teams can agree on the `IDamageCalculator` interface and proceed to build their respective components independently, confident that as long as the final implementation adheres to the agreed-upon contract, integration will be seamless [4].

There are several well-established mechanisms for injecting dependencies into a class. The most common and recommended is Constructor Injection, where dependencies are supplied through a class's constructor parameters [4] [5]. This makes dependencies explicit and ensures that an object is always created in a valid, fully-initialized state, as all its required services are guaranteed to be present upon construction [14]. For example, a `ProjectileSpawner` class might be defined as `public ProjectileSpawner(IEntityManager entityManager, IPhysicsService physicsService) { ... }`. This immediately communicates to any developer reading the code that the spawner cannot function without an entity manager and a

physics service. Property Injection involves assigning dependencies to public properties, often marked with an annotation to signal the injector [4]. This is useful for optional dependencies that may not be needed in all cases. Method Injection passes dependencies as parameters to specific methods that require them, which is ideal for short-lived dependencies or for resolving ambiguity about which dependency a method should use [5]. The choice of injection method depends on the specific needs of the component, but constructor injection is generally favored for core, required dependencies due to its clarity and strength in enforcing object validity.

The philosophical underpinning of effective dependency management is the Dependency Inversion Principle (DIP), one of the five SOLID principles of object-oriented design [13]. DIP states two things: first, high-level modules (like gameplay rules and combat logic) should not depend on low-level modules (like rendering, input, or physics); both should depend on abstractions (like interfaces) [13]. Second, abstractions should not depend on details; details (concrete implementations) should depend on abstractions [13]. In the context of our combat system, this means the `CombatRules` module (a high-level policy) should never reference a concrete class like `DirectXRenderer` or `PhysXCollider`. Instead, it should depend on an abstract `IRenderer` interface and an `IPhysicsService` interface [22]. The engine-specific implementations of these interfaces, such as `DirectXRenderer` or `OpenGLRenderer`, would then depend on the `IRenderer` abstraction for compilation [13]. This "inversion" of the typical top-to-bottom dependency flow is what makes the architecture resilient. If the project decides to switch from DirectX to Vulkan, only the `VulkanRenderer` class needs to be written or modified. The entire `CombatRules` module remains completely untouched because it only ever interacts with the `IRenderer` interface, which remains stable [22].

This principle of abstraction extends beyond engine subsystems to the internal logic of the combat system itself. The `DamageApplicationSystem` should not know or care whether it is dealing damage to a player, an NPC, or a destructible environment object. It should operate on an abstract `IDamageable` interface. Any entity that can take damage—be it a `PlayerCharacter` or a `Barrel`—would implement this interface. The system queries for all entities that conform to the `IDamageable` contract and applies its logic generically. This eliminates the need for `is` checks or casting, which are classic signs of tight coupling and violation of DIP [44]. Patterns like the Adapter Pattern can be used to make existing classes compatible with these new abstractions without modifying their source code [3] [12]. For instance, if a legacy `OldEnemyAI` class exists, an adapter can be created that implements the `IDamageable` interface and translates calls to it into the necessary interactions with the `OldEnemyAI` object.

The practical implementation of DI often involves a DI container, a framework that automates the process of object creation and wiring [5]. These containers, such as Microsoft.Extensions.DependencyInjection for .NET or Spring for Java, maintain a registry of services and their lifetimes (e.g., singleton, scoped, transient) [5]. At application startup, a "Composition Root" is established—a single, centralized location where all the necessary services are registered and the overall object graph is constructed [14]. For example, the Composition Root would register `typeof(IDamageCalculator)`, `typeof(DamageCalculator)` and then build the `ServiceProvider` [9]. From this point forward, whenever a component requests a dependency, the container resolves it from its registry. This greatly simplifies client code, as developers no longer need to manually instantiate and pass around a web of dependencies [9]. However, for smaller projects or simpler cases, a manual approach known as "Poor Man's DI" can suffice, where a parameterless constructor creates dependencies internally [14]. While less flexible than a full-fledged container, it still captures the core benefit of separating construction from usage.

However, DI is not a silver bullet and should be applied judiciously. Overusing DI can lead to "constructor hell," where a class has an unwieldy number of dependencies passed into its constructor [6]. A general guideline is to keep the number of constructor arguments low, ideally around four [14]. If a class requires more dependencies, it is often a strong indicator that it has violated the Single Responsibility Principle and has become too big and complex. Such a class should be refactored into smaller, more focused components, each with its own clear responsibility and fewer dependencies [14]. Furthermore, not all objects require DI. Simple value types, primitive data transfer objects (DTOs), and utility classes that have no external dependencies or side effects can and should be instantiated directly using the `new` keyword [14]. The rule of thumb is to apply DI to services and components that represent external dependencies, such as database access, network communication, or, in our case, engine-specific subsystems [14]. By thoughtfully applying DI and adhering to DIP, the combat system becomes a collection of loosely coupled, highly cohesive modules that communicate through well-defined interfaces, forming a robust and portable architecture ready to be integrated with any engine or platform.

Feature	Traditional Approach (Creation Inside)	Dependency Injection (Usage Outside)
Control Flow	Client creates its own dependencies.	Dependencies are provided to the client by an external source (the injector).
Coupling	High coupling. Client is tightly coupled to the concrete implementation of its dependencies.	Low coupling. Client depends only on an abstraction (interface), not a concrete class [4].
Testability	Difficult. Testing requires the real, concrete dependencies, which may involve slow I/O or external services.	Excellent. Dependencies can be easily replaced with mock or stub implementations for isolated unit testing [5].
Flexibility	Rigid. Changing a dependency requires modifying the client's source code.	Highly flexible. Different implementations of a dependency can be swapped in at runtime without changing the client code [13].
Code Clarity	Responsibilities are mixed. Creation logic is interwoven with business logic.	Clear separation of concerns. Business logic is separate from object construction. Dependencies are explicit [14].

By embracing this inverted model of control, the combat system is built on a foundation of interfaces and abstractions. This ensures that the core logic of how combat works is preserved, regardless of whether the system is running on a PC, a console, or a mobile device. The renderer, the input handler, the audio engine—all are treated as configurable services that can be swapped out. This architectural purity is the definitive path to achieving true engine-agnosticism.

Communication Without Coupling: Event Systems and Gameplay Tags

In a complex, modular system like a combat engine, components must be able to react to and influence one another without developing direct, hard-wired dependencies. Direct calls, where one system instantiates and invokes a method on another, create tight coupling that undermines the entire purpose of an engine-agnostic architecture [41]. If the `CombatManager` directly calls a method on the `ParticleEffectManager`, any change to the particle manager's API could ripple through and break the combat manager. To solve this problem, the system must employ mechanisms for indirect, asynchronous communication. The two most powerful tools identified in the research for achieving this are event-driven architectures and Gameplay Tags. Events allow systems to broadcast announcements about state changes, while tags provide a standardized way to label and categorize entities and states, enabling broad, decoupled queries [41] [43]. Together, they form a robust communication layer that allows the combat system to remain flexible, scalable, and entirely free of embedded lore.

An event-driven architecture, specifically a publish/subscribe (pub/sub) model, is a design pattern where publishers and subscribers communicate indirectly [41]. A publisher is a component that generates an event to signal that something of interest has happened (e.g., a unit was defeated, an ability was cast). A subscriber is a component that registers its interest in specific types of events and defines a callback function (an event handler) to be executed when one of those events is published [41]. This model inherently decouples the source of an action from its consequences. The combat system doesn't need to know who will react to a critical hit; it only needs to publish a `CriticalHitEvent`. Subscribers, such as a UI system that displays a visual effect, a sound system that plays a special audio cue, and an achievement system that checks for a "land a critical hit 100 times" milestone, can all react independently [41]. This loose coupling is achieved because neither the publisher nor the subscribers hold direct references to each other's concrete implementations. They only interact with a central `EventBus` or dispatcher that manages subscriptions and routing [41].

For an event system to be effective in a high-performance, engine-agnostic context, several design considerations are crucial. First, the identification of events must be fast and efficient. Using string names like `"OnUnitDefeated"` for every event check is slow and prone to typos. A superior approach is to use a hashing function like Murmur3 to generate a stable, unique 32-bit integer ID from the event's string name at compile time [39]. This allows for near-instantaneous lookups in hash maps, significantly improving dispatch performance [39]. Second, event payloads—the data carried by the event—must be designed for flexibility and reuse. Instead of defining a massive, monolithic event struct with every possible field, a more modern ECS-inspired approach is to use payload "components" [39]. An event acts as an entity container holding a collection of self-contained payload objects. For a `CollisionEvent`, the payload might contain a `CollisionPayload` component with the involved entity IDs and a displacement vector, but nothing else [39]. This avoids the "fat event" problem, where an event carries unnecessary data, and allows the same event to be reused in different contexts with only the relevant payload components attached. Third, to ensure deterministic behavior, the system must define a clear ordering for event processing. This can be achieved by assigning priorities to events themselves (e.g., input events fire before gameplay events) and to their handlers (to resolve conflicts when multiple handlers subscribe to the same event type) [39].

While events are excellent for broadcasting *occurrences*, they are less effective for querying *states*. To determine if an entity is currently "on fire," a system might need to poll it repeatedly. This is where Gameplay Tags excel. Gameplay Tags are a system for assigning string-based labels to entities or states, which offer significant advantages over

traditional enumerations (enums) for dynamic gameplay systems [42]. Unlike enums, whose values are fixed at compile time, tags can be defined and managed externally, even by designers in config files or editor interfaces, allowing the system to adapt to new content without code changes [43]. They support a hierarchical structure, meaning a tag like `Status.Burning` can be a child of a parent tag `Status`. This allows systems to perform broad queries, such as checking for the presence of any `Status` tag, without needing to know the specific status of the entity [43] [44].

The power of Gameplay Tags lies in their ability to completely abstract away narrative details and tie them to generic gameplay logic. For example, a mechanic that applies "bleeding damage" can be implemented by checking for the presence of a `Status.Bleeding` tag on a target entity. The logic for applying the damage is completely agnostic to what caused the bleed—in-game, it could be a dagger wound, a poison, or a magical effect. The system only cares that the tag is present. This perfectly fulfills the requirement of zero lore embedding. The narrative of *why* the entity is bleeding is irrelevant to the mechanical *process* of taking damage. This decoupling is further enhanced by the ability for different modules to define and use the same tags without having a direct code dependency on each other [42]. The `combat` module and the `faction` module can both safely use a `Faction.Player` tag, as long as they share a common definition of the tag string [42].

Gameplay Tags can be used to drive nearly every aspect of the combat system, from ability triggers to environmental interactions. Ability validity can be determined by a system that checks an ability's `TriggerValidityData` component against tags on the caster and target [20]. A weapon's description might be stored in a data file linked to a `WeaponType.Sword` tag, while its gameplay properties (damage, speed) are defined in a separate, generic `WeaponStats` table. The combat logic uses the tag to find the stats, but the tag itself contains no lore [10]. The hierarchical nature of tags also allows for elegant filtering. A loot table could use a `RequiredTags` container to specify that an item can only be dropped if the defeated enemy has both the `Enemy.Type.Goblin` and `Enemy.Rarity.Elite` tags, and a `BlockedTags` container to ensure the item isn't dropped if the enemy has the `Status.Undead` tag [44]. This provides a powerful, designer-driven system for content generation that is entirely divorced from C++ or C# code. The Unreal Engine's implementation of GameplayTags, with its fast FName-backed storage and hierarchical matching, serves as a strong conceptual model for how such a system can be built and scaled [42] [44].

Aspect	Traditional Enums	Gameplay Tags
Definition	Defined in source code, values are fixed at compile time.	Can be defined externally in data files or editor, allowing runtime addition/modification [43].
Coupling	Tight coupling. Adding a new enum value requires recompiling dependent code.	Loose coupling. Modules can use tags without depending on each other's code [42].
Hierarchy	Flat structure. Limited ability to group related values.	Supports parent-child relationships (e.g., `Action.Primary`, `Action.Secondary`) [43] [44].
Use Case	Best for small, static sets of options deep within system code (e.g., cardinal directions) [42].	Ideal for representing dynamic game states, abilities, factions, and events [43].
Designer Access	Not accessible to designers without modifying source code.	Designed to be editable by designers via configuration files or in-editor tools [43].
Performance	Very fast comparisons.	Comparisons are comparable to FNames, considered very fast [42].

By combining a high-performance pub/sub event system with a flexible, hierarchical tagging system, the combat architecture achieves a remarkable degree of decoupling. Events handle the broadcast of discrete occurrences, while tags provide a persistent, queryable state. This layered communication model ensures that the system's logic remains clean and focused, reacting to abstract signals rather than concrete objects or narrative facts. It is the ultimate expression of the principle of separation of concerns, allowing each subsystem—from damage calculation to particle effects—to evolve and be developed independently, all while participating in a coherent and responsive combat simulation.

Data Management and Serialization Strategies

The principle of a combat system with zero lore embedded in code is fundamentally a data-driven design (DDD) methodology [7]. This means that the vast majority of the system's behavior, parameters, and content—including the rules of engagement, the properties of weapons, the attributes of enemies, and the effects of abilities—must reside in external data files rather than being hardcoded into the application [12]. The engine's job is to provide a generic set of rules and mechanisms for interpreting and executing this data; the specific "what to do" is dictated entirely by the loaded data files [7]. To accomplish this, a robust strategy for data management, encompassing data format selection, schema validation, and serialization/deserialization, is paramount. This section will explore the critical choices and techniques involved in managing external data,

focusing on how to ensure data integrity, optimize performance, and maintain the strict separation between code and content.

The choice of data format is the first and most consequential decision. Human-readable formats like JSON (JavaScript Object Notation) and YAML (YAML Ain't Markup Language) are typically preferred during the development phase [2] [7]. Their syntax is easy for humans to read, write, and debug, making them ideal for iterative design workflows where level designers or game designers might need to tweak values directly [7]. For instance, a weapon's damage profile could be defined in a `weapon.json` file, allowing a designer to change the damage value from `50` to `75` without requiring a developer to recompile the game [10]. However, these formats have drawbacks for production builds. They tend to result in larger file sizes and slower parsing speeds compared to binary formats [2]. For performance-critical applications like a combat system that might need to load hundreds of asset definitions at startup, these differences can be significant.

Binary formats, on the other hand, offer substantial advantages in terms of performance and size [2]. They produce compact files that can be loaded and parsed much faster, reducing initial loading times and minimizing memory footprint. For an engine-agnostic system, this is a key consideration for ensuring consistent performance across different hardware targets. A common strategy is to adopt a hybrid approach: using human-readable formats during development and employing a build-time pipeline to convert these assets into a highly optimized, compact binary representation [1]. This build process can read the original JSON or YAML files and write them into a custom binary blob format. This allows the development team to enjoy the productivity benefits of editable text files while shipping a product-optimized format to end-users [1]. This conversion can even happen at runtime, as demonstrated by a system that reads a JSON file and writes its contents directly into a binary blob, which can then be used for immediate, fast access [1].

Once a data format is chosen, the next challenge is ensuring data integrity. External data must never be trusted implicitly; it must be rigorously validated before it is used by the game [7]. Invalid or malformed data can lead to crashes, exploits, or subtle bugs that are difficult to diagnose. Schema validation is the primary defense against this. Tools like JSON Schema can be used to define the expected structure, data types, and constraints for each data file (e.g., a `health` value must be a positive integer, a `weaponName` must be a non-empty string) [7]. Before loading any data, the system should validate it against its corresponding schema. If the data fails validation, the loading process should fail gracefully, logging an error that points to the specific file and line number of the

problem, preventing corrupted data from corrupting the game state [7]. This practice of "never trusting external data" is a cornerstone of secure and robust software engineering [7].

The process of converting data between its serialized (file-based) representation and its in-memory object form is called serialization (saving/loading) or deserialization (loading). There are two primary approaches: automatic and manual serialization. Automatic serialization, often facilitated by libraries or reflection, can generate the necessary save/load code automatically based on the class definition. This approach is generally faster for development and is suitable for most use cases, with a typical performance difference of 2-10x being negligible during loading screens [2]. Manual serialization, where a developer writes the `Save` and `Load` methods for each class by hand, offers maximum control and can be pushed to the absolute limits of performance, which might be necessary for dynamically loading data at runtime or for minimizing save file sizes [2]. A hybrid approach is often the most pragmatic solution: using an automatic serializer for most classes while allowing a few critical, performance-sensitive classes to override it with custom, optimized `Load`/`Save` methods [2].

A significant technical hurdle in serialization arises when dealing with complex object graphs that contain shared references [11]. For example, consider a `Player` object that has a reference to a `Sword` object. Both the `Player` and the `Sword` might also hold a reference back to the `Player`. If a standard serializer is used, it will encounter the `Player` object once when serializing the `Player`'s `sword` field, and then again when serializing the `Sword`'s `owner` field. It will attempt to serialize the entire `Player` object a second time, creating a duplicate copy [11]. Upon deserialization, this results in two separate `Player` objects that are logically the same but physically distinct, leading to a bug known as the "Multiplying Wives Problem" [11]. The solution is to teach the serializer about object identity. This can be achieved by implementing a system that tracks all serialized objects and, when it encounters a reference to an already-serialized object, instead of writing the object again, it writes a small token or offset that represents a pointer to the original object [1]. Open-source libraries like SharpSerializer are specifically designed to handle this issue correctly, preserving the original reference structure [11]. Another advanced technique involves creating a "memory blob," a contiguous block of memory that stores the entire serialized object graph. Within this blob, all pointers are converted into "Relative Pointers" that store an offset from the start of the blob, allowing the entire structure to be loaded and used immediately without any post-loading patching-up process [1].

Finally, the concept of a persistent data asset is central to a data-driven approach. In engine-agnostic terms, this is simply a data structure that can be saved to disk and loaded back into memory. In engine-specific contexts, this is often implemented using assets like Unity's ScriptableObjects [10]. ScriptableObjects are C# classes that inherit from a base class, allowing them to be saved as unique asset files in the project folder. They are ideal for storing static, shared data like item definitions, enemy statistics, or character dialogue, as they have a smaller memory footprint than MonoBehaviours and can be referenced by multiple game objects, implementing the Flyweight pattern [10]. Critically, they can be extended with editor tools, allowing designers to create and modify these data assets visually in the engine's inspector [8]. While using ScriptableObjects ties the data model to Unity, the underlying principle is portable. An engine-agnostic equivalent would be a custom binary format or a database schema that defines the structure of these static game assets. The key takeaway is the separation: the gameplay logic (the code) knows how to consume a generic `ItemDefinition` object, but it does not know how that object is created or stored. This separation is the essence of maintaining zero lore in the code and ensuring the system remains flexible and portable [10].

Implementation Blueprint and Code-Level Guidance

Translating the theoretical principles of an engine-agnostic, lore-free combat system into a concrete implementation requires a disciplined approach to code organization, data structure design, and architectural enforcement. This section provides a practical blueprint, outlining a sample component structure, a set of representative systems, and the specific coding patterns that enforce the desired architectural constraints. The guidance focuses on creating a system where gameplay logic is expressed through data and interactions, not through hardcoded narrative or engine-specific calls. Every recommendation is grounded in the patterns discussed previously: Entity-Component-System (ECS) for modularity, Dependency Injection (DI) for decoupling, and Gameplay Tags for abstraction.

The foundation of the implementation is the definition of pure data components. These structs should contain only state, with no associated functions [16] [19]. They represent atomic pieces of an entity's state. For a combat system, a minimal viable set of components might include:

Component Name	Purpose	Example Fields
`EntityId`	A unique identifier for the entity. Often a simple integer or pointer [16].	`id: u64`
`TagContainer`	A collection of Gameplay Tags that describe the entity's properties, affiliation, and status [43] [44].	`tags: HashSet<GameplayTag>`
`Health`	Contains current and maximum health values.	`current: f32, max: f32`
`Position`	The entity's world coordinates.	`x: f32, y: f32, z: f32`
`Velocity`	The entity's movement vector.	`x: f32, y: f32, z: f32`
`AbilityData`	Defines the properties of an ability, including damage, cooldown, and targeting rules.	`damage: f32, cooldown: f32, targetType: TargetType`
`DynamicBuffer<T>`	A buffer-like component for storing a sequence of items, such as events or impacts [20].	`items: Vec<T>`

With these data components defined, the next step is to implement the systems. Systems are functions or classes that iterate over entities possessing the required components [15]. They embody the game's logic. Here is a blueprint for a few core combat systems:

1. **DamageApplicationSystem**: This system is responsible for applying damage to targets.

- **Query**: Iterates over all entities with a `Health` component.
- **Logic**: Listens for a `DamageAppliedEvent`. The event's payload contains the target `EntityId`, the damage amount (`f32`), and potentially a `GameplayTag` representing the damage type (e.g., `DamageType.Physical`). The system then iterates through its list of `Health` components, finds the one matching the target's ID, and subtracts the damage amount. If the resulting health drops to zero or below, it publishes a `UnitDefeatedEvent` with the target's ID in its payload [41].

1. **AbilityExecutionSystem**: This system governs how abilities are triggered and resolved.

- **Query**: Listens for events indicating an ability has been activated (e.g., a `PlayerAbilityInputEvent`).
- **Logic**: It retrieves the `AbilityData` component of the casting entity. Based on this data, it determines the ability's effect. For a projectile ability, it might create a new entity with a `Position`, `Velocity`, and `Projectile` component. For an area-of-effect ability, it might query the spatial index for all entities within a radius and publish a `DamageAppliedEvent` for each one found [20]. The state of the ability,

like a cooldown, would be stored in a `CooldownTimer` component attached to the caster, demonstrating persistence without a central authority [20].

1. **MovementSystem**: This system handles the physics and motion of combatants.

- **Query**: Iterates over all entities with both `Position` and `Velocity` components.
- **Logic**: On each frame/tick, it adds the `Velocity` vector to the `Position` of each entity. This simple transformation drives all movement in the simulation, from player characters to falling debris.

To enforce the separation of mechanics and narrative, the use of Gameplay Tags must be pervasive. Any conditional logic based on an entity's state should use tags, not class inheritance or hardcoded strings. For example, instead of writing `if (enemy.GetType() == Goblin.class) { dropGold(); }`, the logic should be: `if (entity.HasTag("Enemy.Type.Goblin")) { DropGold(); }`. The condition is now data-driven and can be altered by editing a data file that assigns the `Enemy.Type.Goblin` tag to an entity, without touching the code that checks for it [44]. This extends to all aspects of the game, from determining which sound to play on impact (`GameplayCue.Weapon.Impact.Metal`) to defining compatibility for equipment slots (`Robot.Slot.Weapon.Laser`) [43]. The naming convention itself, using dots for hierarchy (e.g., `Event.Combat.Attack`), is a deliberate choice that supports both readability and powerful pattern-matching queries [43].

Enforcing engine-agnosticism at the code level requires a strict discipline of abstraction. Every interaction with a non-core, engine-specific subsystem must go through an abstract interface. The following interfaces should be defined in a core, engine-agnostic library:

- `interface ITimeProvider { float GetDeltaTime(); }`
- `interface IInputReader { InputState ReadInput(); }`
- `interface IAudioService { void PlaySound(AudioClip clip, Position pos); }`
- `interface IRenderService { void DrawMesh(MeshData mesh, Position pos); }`
- `interface IAssetLoader { T Load(string assetPath); }`

The core ECS world and all its systems would be constructed using these interfaces, never the concrete implementations. At the application's entry point, a "Composition Root" would be established. This is the single place where the DI container is used to wire everything up [14]. Here, the concrete implementations are mapped to their interfaces:

```
// Pseudo-code for a Composition Root
serviceCollection.AddSingleton<ITimeProvider, GameTimeProvider>(); // Uses
serviceCollection.AddSingleton<IInputReader, WindowsInputReader>(); // Use
```

```
serviceCollection.AddSingleton<IAudioService, XAudio2Service>(); // Uses e
// ... etc
var serviceProvider = serviceCollection.BuildServiceProvider();

// The core engine is handed the provider and resolves its dependencies
var gameEngine = new GameEngine(serviceProvider);
```

This approach, inspired by the use of a DI container to manage system initialization, ensures that the core logic is completely insulated from the specifics of the platform [9]. If the project were ported to a different engine, only the implementations of `IInputReader`, `IAudioService`, etc., would need to be rewritten. The `GameEngine` and all its internal systems would remain unchanged [22].

Finally, to maintain the integrity of this architecture over time, automated enforcement through static analysis is highly recommended. Custom linter rules can be written to scan the codebase and flag violations of the architectural rules [35]. For example, a custom lint rule could be created to detect any direct instantiation of an engine-specific class (e.g., `new GameObject()` or `new Texture2D()`) and report it as an error. Another rule could search for string literals that match a narrative pattern (e.g., "fireball", "orc warrior") inside gameplay logic and warn the developer that this might be embedding lore. This transforms architectural compliance from a matter of developer discipline into an automated, repeatable process, catching regressions early in the development cycle [34]. By combining a clear component-and-system blueprint with rigorous interface-based programming and automated validation, it is possible to construct a combat system that is not only technically robust but also architecturally pristine and perfectly aligned with the goals of engine-agnosticism and narrative neutrality.

Synthesis and Strategic Considerations

The successful implementation of a combat system that is both engine-agnostic and free of embedded lore hinges on the synthesis of several powerful architectural principles: Data-Oriented Design (DOD), the Entity-Component-System (ECS) pattern, strict abstraction through interfaces, and a deeply integrated data-driven approach. This final section synthesizes the preceding analysis into a cohesive architectural vision and addresses the strategic considerations, challenges, and inherent trade-offs involved in pursuing such a design. The resulting blueprint is not merely a set of guidelines but a

holistic philosophy for building games, where the malleability of data supersedes the rigidity of code, and portability is a feature engineered from the ground up.

The core architectural vision presented throughout this report is one where the combat system is fundamentally a processor of data. The ECS pattern provides the organizational framework for this vision, separating identity (Entities), pure state (Components), and behavior (Systems) [15] [16]. This separation is the key to fulfilling both primary requirements. For engine-agnosticism, it forces all logic into systems that operate on abstract data structures. These systems are then wired together using Dependency Injection (DI), where they depend only on interfaces like `IDamageable` or `IPhysicsService`, not concrete engine implementations [4] [22]. This inversion of control, guided by the Dependency Inversion Principle (DIP), ensures that the high-level gameplay logic is completely insulated from the volatile details of rendering APIs, input drivers, or audio engines [13]. Porting the system to a new platform thus becomes a task of swapping out implementations of these interfaces, a far simpler endeavor than rewriting core gameplay logic.

For the requirement of zero lore embedding, the same ECS foundation provides the answer. Because behavior is defined by systems acting on data, and that data is loaded from external sources, the connection between narrative and code is severed [7]. The `DamageApplicationSystem` does not know or care *why* an entity is taking damage; it simply modifies the `Health` component of any entity that possesses it. The reason is encoded in the entity's `TagContainer` component, which might contain tags like `Status.Bleeding` or `Cause.ByFire` [44]. This use of Gameplay Tags is the linchpin of this decoupling. They provide a universal, extensible vocabulary for describing gameplay states and events, allowing systems to react to abstract concepts rather than hardcoded story elements [42] [43]. A mechanic that applies "poison damage" is implemented by looking for the `Status.Poisoned` tag, a logic that is entirely independent of whether the poison was delivered by a snake bite, a spell, or a tainted blade. This transforms game design from a process of writing code to a process of configuring data, empowering designers and keeping the source code focused on generic, reusable systems.

However, this powerful and flexible architecture comes with significant strategic considerations and inherent trade-offs. The most prominent challenge is the increased complexity of implementation. Building a performant, feature-rich ECS framework from scratch is a substantial engineering undertaking [18]. It requires careful attention to memory management, parallel execution safety, and robust serialization to handle complex object graphs [11]. While the principles are portable, the implementation details are not trivial. Furthermore, the benefits of engine-specific tooling, such as Unity's rich

editor integration for ScriptableObjects, must be replicated [8] [10]. A purely engine-agnostic system would require the development of custom authoring tools to allow designers to effectively create and manage the data assets, adding another layer of development effort.

Another trade-off exists between data locality and abstraction. While DOD champions the use of flat, contiguous arrays for performance, traditional OOP abstractions like inheritance hierarchies and virtual function calls, which enable polymorphism, rely on indirection through pointers [37]. Following these pointers to access object data can lead to scattered memory access and cache misses, undermining the performance benefits of DOD [37]. This does not mean that all abstraction is bad, but it suggests that a careful balance must be struck. Polymorphism should be used judiciously, favoring composition and data-centric approaches where possible. For example, instead of having a base `Attack` class with virtual `CalculateDamage()` methods for different subclasses (`MeleeAttack`, `RangedAttack`), the system would be better served by having a single `DamageCalculationSystem` that queries for entities with both an `AttackComponent` and a `DamageProfileComponent`, and then uses the data within those components to compute the result. This avoids pointer chasing and leverages the cache-friendly iteration of ECS [37].

The choice of serialization format also presents a strategic decision. While binary formats offer superior performance, they sacrifice the human readability and editability of formats like JSON [2]. A project must weigh the development-time productivity gained from easy-to-edit text files against the runtime performance benefits of a binary representation. The recommended compromise is a build-time conversion pipeline that bridges this gap, but this adds complexity to the asset processing workflow [1]. Similarly, while Gameplay Tags are incredibly powerful, they introduce their own form of indirection. Querying for a tag requires a lookup operation, which is fast but not instantaneous. For extremely hot paths where every nanosecond counts, a system might opt for a more direct data structure, accepting a degree of rigidity in exchange for raw speed. However, for the vast majority of combat logic, the flexibility afforded by tags far outweighs this minor performance cost.

In conclusion, the path to creating an engine-agnostic combat system with zero lore embedded in code is paved with the principles of data orientation and strict decoupling. By adopting an ECS architecture, leveraging interfaces and Dependency Injection, and embracing a data-driven model powered by Gameplay Tags, it is possible to construct a system of remarkable flexibility, portability, and maintainability. This approach fundamentally shifts the locus of control from the rigidity of code to the fluidity of data.

While the initial investment in building the supporting infrastructure and tooling is significant, the long-term benefits—a system that is easier to tune, extend, and port—are substantial. It represents a forward-thinking approach to game architecture, one that prioritizes modularity and separation of concerns to build games that can adapt and thrive across diverse platforms and over long development cycles.

Reference

1. Serialization For Games | Gabriel's Virtual Tavern https://jorenjoestar.github.io/post/serialization_for_games/
2. Save/Load and game performance - Automatic or manual ... https://discussions.unity.com/t/save-load-and-game-performance-automatic-or-manual-serialization/622048
3. Facade and Adapter Design Patterns Explained https://codesignal.com/learn/courses/backward-compatibility-in-software-development-1/lessons/facade-and-adapter-design-patterns-explained
4. Design Patterns Explained – Dependency Injection https://stackify.com/dependency-injection/
5. Dependency injection https://en.wikipedia.org/wiki/Dependency_injection
6. Ways of keeping configuration code out of logic code using ... https://stackoverflow.com/questions/7288859/ways-of-keeping-configuration-code-out-of-logic-code-using-dependency-injection
7. [Share] Data-Driven Design: Leveraging Lessons from ... https://dev.to/methodox/data-driven-design-leveraging-lessons-from-game-development-in-everyday-software-5512
8. Building a Data-Driven Combo System in Unity - Funder Games https://funder-games.com/news/building-a-data-driven-combo-system-in-unity/
9. Dependency Injection in Game Dev https://dev.to/thebuzzsaw/dependency-injection-in-game-dev-1hdh
10. Separate Game Data and Logic with ScriptableObjects https://unity.com/how-to/separate-game-data-logic-scriptable-objects
11. Tips on Game World State Data Serialization in Unity C# ... http://www.archmagerises.com/news/2015/9/22/tips-on-game-world-state-data-serialization-in-unity-c

12. Game Programming Patterns https://medium.com/@apoorvmishra05/game-programming-patterns-9e4a877dc942
13. Dependency inversion principle https://en.wikipedia.org/wiki/Dependency_inversion_principle
14. c# - Dependency Injection and development productivity https://stackoverflow.com/questions/10313507/dependency-injection-and-development-productivity
15. What is an ECS and Why Rust? A Deep Dive into Data- ... https://medium.com/@jordangrilly/what-is-an-ecs-and-why-rust-a-deep-dive-into-data-oriented-game-engine-design-887680a5583a
16. Entity component system https://en.wikipedia.org/wiki/Entity_component_system
17. Entity Component System FAQ https://github.com/SanderMertens/ecs-faq
18. The Essence of Entity Component System https://arxiv.org/html/2606.14919v1
19. Deep-diving into Entity Component System (ECS) Architecture ... https://prdeving.wordpress.com/2023/12/14/deep-diving-into-entity-component-system-ecs-architecture-and-data-oriented-programming/
20. Gameplay Ability System in a data oriented approach https://discussions.unity.com/t/gameplay-ability-system-in-a-data-oriented-approach/869479
21. Separating game data/logic from rendering https://gamedev.stackexchange.com/questions/62980/separating-game-data-logic-from-rendering
22. Part 2: Decoupling the Renderer: Terminal to Raylib in ... - Lorbic https://lorbic.com/decoupling-game-engine-renderer-raylib/
23. Do you decouple animation code from gamecode? https://discussions.unity.com/t/do-you-decouple-animation-code-from-gamecode/1545963
24. Devlog #2: game engine architecture - Dom Williams https://domwillia.ms/devlog2/
25. Game Engine Architecture: Systems Design & Patterns 2025 https://generalistprogrammer.com/game-engine-architecture
26. C++ Game Engine with Data-Driven Design and Slay the Spire ... https://www.linkedin.com/posts/samay-thusoo-09m2019_gamedev-cpp-gameengine-activity-7457566724846948353-nvZT
27. Looking for Data Oriented Games (no ECS) : r/gamedev https://www.reddit.com/r/gamedev/comments/1i1et60/looking_for_data_oriented_games_no_ecs/
28. SoA vs AoS: Data Layout Optimization https://www.abhik.ai/concepts/systems/soa-vs-aos
29. Vittorio Romeo's Post https://www.linkedin.com/posts/vittoriomeo_aos-vs-soa-in-practice-particle-simulation-activity-7340377524880711680-Qucm

30. 1 Understanding data-oriented design https://www.manning.com/preview/high-performance-unity-game-development/chapter-1
31. Phantom Types - dsp https://experimentalworks.net/posts/2024-01-22-simple-phantom-types/
32. All the same thing? ADTs, sum types, discrimnated unions ... https://www.reddit.com/r/functionalprogramming/comments/qi8drg/all_the_same_thing_adts_sum_types_discrimnated/
33. Considering SystemVerilog Tagged Unions https://www.prbs23.com/blog/posts/considering-systemverilog-tagged-unions/
34. Static Code Analysis: Top 7 Methods, Pros/Cons and Best ... https://www.oligo.security/academy/static-code-analysis
35. Static analysis with Custom Lint Rules (Android Dev Summit '19) https://www.youtube.com/watch?v=jCmJWOkjbM0
36. Customizing static analysis https://dart.dev/tools/analysis
37. Data Locality · Optimization Patterns https://gameprogrammingpatterns.com/data-locality.html
38. It's so incredibly difficult to decouple things in game ... https://www.reddit.com/r/godot/comments/ybqzgu/its_so_incredibly_difficult_to_decouple_things_in/
39. Why I Built My Own Event System for the Brokkr Engine https://noctrlstudios.com/posts/why-i-built-my-own-event-system-for-the-brokkr-engine/
40. How to Create Event Payload Design: A Practical Guide https://oneuptime.com/blog/post/2026-01-30-how-to-create-event-payload-design/view
41. Nomad Game Engine: Part 7— The Event System https://medium.com/@savas/nomad-game-engine-part-7-the-event-system-45a809ccb68f
42. Can Gameplay Tags be used instead of enumerations? https://www.reddit.com/r/unrealengine/comments/1gss7n5/can_gameplay_tags_be_used_instead_of_enumerations/
43. Gameplay Tag Best Practices https://unrealdirective.com/resources/gameplay-tags/best-practices/
44. Why you should be using GameplayTags in Unreal Engine https://tomlooman.com/unreal-engine-gameplaytags-data-driven-design/